

Core Java

Complete Learning Material — From Scratch to Associate Software Engineer (Full Stack)

L1 & L2 Interview Preparation Guide

Target Role	Associate Software Engineer — Full Stack
Interview Levels	L1 (Technical Screening) & L2 (Technical Deep-Dive)
Prerequisites	None — starts from scratch
Covers	Java Basics, OOP, Collections, Multithreading, Streams, JDBC, JVM
Style	Concepts + Code + Interview Q&A

Table of Contents

1. Introduction to Java & the JVM
2. Setup, First Program & Compilation Flow
3. Data Types, Variables & Operators
4. Control Flow Statements
5. Arrays & Strings
6. Methods & Method Overloading
7. Object-Oriented Programming (OOP)
8. Inheritance, Polymorphism, Abstraction, Encapsulation
9. Interfaces vs Abstract Classes
10. Packages & Access Modifiers
11. Exception Handling
12. Collections Framework (List, Set, Map, Queue)
13. Generics
14. Multithreading & Concurrency
15. Java 8+ Features (Lambdas, Streams, Optional, Functional Interfaces)
16. File I/O & Serialization
17. JDBC — Database Access
18. JVM Internals, Memory Model & Garbage Collection
19. L1 Interview Questions (50)
20. L2 Interview Questions (40)
21. Coding Problems for Practice
22. Quick Revision Cheat Sheet

1. Introduction to Java & the JVM

Java is a class-based, object-oriented, platform-independent programming language created by James Gosling at Sun Microsystems in 1995 (now owned by Oracle). Its slogan, “**Write Once, Run Anywhere**”, is enabled by the Java Virtual Machine (JVM).

Key Features

- **Platform Independent** — Java source is compiled to bytecode (.class), which runs on any JVM.
- **Object-Oriented** — Everything is built around classes and objects.
- **Robust & Secure** — Strong type-checking, exception handling, no explicit pointers.
- **Multithreaded** — Built-in support for concurrent programming.
- **Automatic Memory Management** — Garbage Collector reclaims unused objects.
- **Rich API** — Huge standard library (Collections, IO, Networking, Concurrency, etc.).

JDK vs JRE vs JVM

Component	Purpose
JDK (Java Development Kit)	Full development kit — compiler (javac), JRE, debugger, tools.
JRE (Java Runtime Environment)	Runtime only — JVM + core libraries to run Java programs.
JVM (Java Virtual Machine)	Loads, verifies and executes bytecode. Platform-specific.

2. Setup, First Program & Compilation Flow

Install the JDK (Oracle JDK or OpenJDK 17/21 LTS recommended). Verify with `java -version` and `javac -version`. Use an IDE like IntelliJ IDEA, Eclipse, or VS Code.

First Program — Hello World

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Compile & Run:

```
javac Hello.java // produces Hello.class (bytecode)  
java Hello      // JVM executes the bytecode
```

Compilation Flow

- **Source** (.java) → **javac** → **Bytecode** (.class)
- Bytecode → **JVM Class Loader** → loaded into memory
- **Bytecode Verifier** checks safety
- **Execution Engine** (Interpreter + JIT compiler) runs the code

3. Data Types, Variables & Operators

Primitive Data Types (8)

Type	Size	Default	Range / Example
byte	1 byte	0	-128 to 127
short	2 bytes	0	-32,768 to 32,767
int	4 bytes	0	-2 ³¹ to 2 ³¹ -1
long	8 bytes	0L	-2 ⁶³ to 2 ⁶³ -1
float	4 bytes	0.0f	~7 decimal digits
double	8 bytes	0.0d	~15 decimal digits
char	2 bytes	'\u0000'	Unicode character
boolean	1 bit	false	true / false

Non-Primitive (Reference) Types

- **String** — immutable sequence of characters.
- **Arrays** — fixed-size collection of same type.
- **Classes & Interfaces** — user-defined types.

Variable Types

- **Local variables** — declared inside methods, must be initialized.
- **Instance variables** — declared in class, belong to objects.
- **Static variables** — declared with `static`, shared across all objects.

Operators

- **Arithmetic:** + - * / %
- **Relational:** == != > < >= <=
- **Logical:** && || !
- **Bitwise:** & | ^ ~ << >> >>>
- **Assignment:** = += -= *= /=
- **Ternary:** condition ? expr1 : expr2
- **instanceof:** tests object type at runtime

4. Control Flow Statements

Conditional

```
if (score >= 90) grade = 'A';
else if (score >= 75) grade = 'B';
else grade = 'C';

switch (day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    default: System.out.println("Other");
}
```

Loops

```
// for loop
for (int i = 0; i < 5; i++) System.out.println(i);

// while loop
int i = 0;
while (i < 5) { System.out.println(i); i++; }

// do-while (executes at least once)
do { System.out.println(i); i++; } while (i < 5);

// enhanced for (for-each)
int[] arr = {1, 2, 3};
for (int n : arr) System.out.println(n);
```

Branching

- **break** — exits a loop or switch.
- **continue** — skips current iteration.
- **return** — exits the method.

5. Arrays & Strings

Arrays

```
int[] arr = new int[5];           // default zeros
int[] nums = {10, 20, 30, 40};    // literal init
int[][] matrix = {{1,2},{3,4}};  // 2D array
System.out.println(arr.length);  // size
```

Strings

Strings in Java are **immutable** — every modification creates a new object. They live in the **String Pool** inside the heap.

```
String s1 = "Hello";              // pooled
String s2 = new String("Hello");  // heap, not pooled
s1 == s2                          // false (different references)
s1.equals(s2)                     // true (same content)

// Common methods
s1.length(); s1.charAt(0); s1.substring(1,4);
s1.toUpperCase(); s1.indexOf('l'); s1.split(",");
String.format("Name=%s Age=%d", "Ravi", 25);
```

StringBuilder vs StringBuffer

Aspect	String	StringBuilder	StringBuffer
Mutable?	No	Yes	Yes
Thread-safe?	Yes (immutable)	No	Yes (synchronized)
Speed	Slow for edits	Fast	Slower than StringBuilder

6. Methods & Method Overloading

```
public int add(int a, int b) { return a + b; }
public double add(double a, double b) { return a + b; } // overloaded
public int add(int a, int b, int c) { return a + b + c; } // overloaded
```

- **Overloading** — same method name, different parameter list (compile-time polymorphism).

- **Pass-by-value** — Java always passes a copy. For objects, the copy is the reference value.
- **varargs** — `void log(String... msgs)` accepts any number of args.

7. Object-Oriented Programming (OOP)

OOP organizes code around **objects** — instances of **classes** — that combine state (fields) and behavior (methods).

Class & Object

```
class Car {
    String brand;
    int speed;

    Car(String brand, int speed) { // constructor
        this.brand = brand;
        this.speed = speed;
    }
    void accelerate() { speed += 10; }
}

Car c = new Car("Honda", 60);
c.accelerate();
```

Four Pillars of OOP

- **Encapsulation** — bundling data and methods; hide internals with `private`; expose via getters/setters.
- **Inheritance** — child class reuses parent class members via `extends`.
- **Polymorphism** — same interface, different behavior (overloading + overriding).
- **Abstraction** — exposing only essential features, hiding implementation (via abstract classes / interfaces).

8. Inheritance, Polymorphism, Abstraction, Encapsulation

```
class Animal {
    void sound() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Bark"); } // overriding
}

Animal a = new Dog(); // upcasting
a.sound();             // prints "Bark" — runtime polymorphism
```

Types of Inheritance

- **Single** — $A \rightarrow B$
- **Multilevel** — $A \rightarrow B \rightarrow C$
- **Hierarchical** — $A \rightarrow B, A \rightarrow C$
- **Multiple inheritance of classes is NOT supported** (Diamond Problem). Achieved via interfaces.

super & this

- **this** — current object reference; used for fields, constructor chaining within same class.
- **super** — parent class reference; call parent constructor or overridden method.

Overloading vs Overriding

Aspect	Overloading	Overriding
Where	Same class	Parent & Child
Signature	Different params	Same signature
Binding	Compile-time	Runtime
Return type	Can differ	Same or covariant

9. Interfaces vs Abstract Classes

```

abstract class Shape {
    abstract double area();           // abstract method
    void describe() { System.out.println("Shape"); }
}

interface Drawable {
    void draw();                     // implicitly public abstract
    default void log() { System.out.println("Drawing..."); } // Java 8+
    static void info() { System.out.println("Drawable v1"); }
}

class Circle extends Shape implements Drawable {
    double r;
    Circle(double r) { this.r = r; }
    double area() { return Math.PI * r * r; }
    public void draw() { System.out.println("Circle"); }
}

```

Aspect	Abstract Class	Interface
Methods	Abstract + concrete	Abstract + default + static + private (Java 9)
Variables	Any type	public static final only
Multiple inheritance	No	Yes (via implements)
Constructor	Yes	No
When to use	Shared base with state	Define a contract / capability

10. Packages & Access Modifiers

Packages organize related classes and prevent naming conflicts. Use `package com.company.module;` at the top of every file.

Modifier	Same Class	Same Package	Subclass (any pkg)	Other
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Non-Access Modifiers

- **static** — belongs to class, not instance.

- **final** — constant variable / non-overridable method / non-extendable class.
- **abstract** — incomplete; must be subclassed/implemented.
- **synchronized** — only one thread at a time in this method/block.
- **volatile** — variable read/write goes directly to main memory.
- **transient** — skip this field during serialization.

11. Exception Handling

An **exception** is an event that disrupts normal program flow. Java exceptions are objects, all descending from Throwable.

Hierarchy

```
Throwable
■■■ Error          (OutOfMemoryError, StackOverflowError) – don't catch
■■■ Exception
    ■■■ Checked    (IOException, SQLException) – must handle / declare
    ■■■ RuntimeException (Unchecked)
        ■■■ NullPointerException
        ■■■ ArrayIndexOutOfBoundsException
        ■■■ ArithmeticException
        ■■■ ClassCastException
```

try-catch-finally

```
try {
    int x = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Arithmetic: " + e.getMessage());
} catch (Exception e) {
    System.out.println("General: " + e);
} finally {
    System.out.println("Always runs");
}
```

throw vs throws

- **throw** — actually throws an exception inside code: `throw new IllegalArgumentException("bad");`
- **throws** — declares that a method may throw a checked exception: `void read() throws IOException`

try-with-resources (Java 7+)

```
try (BufferedReader br = new BufferedReader(new FileReader("a.txt"))) {
    System.out.println(br.readLine());
} // auto-closes the reader
```

Custom Exception

```
class InvalidAgeException extends RuntimeException {
    public InvalidAgeException(String msg) { super(msg); }
}
```

12. Collections Framework

The Collections Framework provides ready-to-use data structures and algorithms. The root interfaces are Collection (List, Set, Queue) and Map (a separate hierarchy).

Hierarchy Overview

```
Iterable
■■■ Collection
    ■■■ List → ArrayList, LinkedList, Vector, Stack
    ■■■ Set  → HashSet, LinkedHashSet, TreeSet
    ■■■ Queue → PriorityQueue, ArrayDeque, LinkedList

Map (separate)
■■■ HashMap
■■■ LinkedHashMap
■■■ TreeMap
■■■ Hashtable, ConcurrentHashMap
```

List

Type	Backing	Order	Duplicates	Best For
ArrayList	Dynamic array	Insertion	Yes	Random access
LinkedList	Doubly linked list	Insertion	Yes	Frequent insert/delete
Vector	Array (synchronized)	Insertion	Yes	Legacy / thread-safe

```
List<String> list = new ArrayList<>();
list.add("A"); list.add("B"); list.add(1, "X");
list.get(0); list.size(); list.remove("A");
Collections.sort(list);
```

Set

- **HashSet** — unordered, $O(1)$ add/contains, allows one null.
- **LinkedHashSet** — maintains insertion order.
- **TreeSet** — sorted (natural / Comparator), $O(\log n)$ operations.

```
Set<Integer> set = new HashSet<>();
set.add(10); set.add(20); set.add(10); // size = 2
```

Map

Type	Order	Null Keys	Thread-safe	Notes
HashMap	None	1	No	$O(1)$ avg; uses hashing + buckets
LinkedHashMap	Insertion	1	No	Useful for caches
TreeMap	Sorted	0	No	Red-black tree, $O(\log n)$
Hashtable	None	0	Yes	Legacy
ConcurrentHashMap	None	0	Yes	Modern concurrent map

```
Map<String, Integer> map = new HashMap<>();
map.put("A", 1); map.put("B", 2);
map.get("A"); map.containsKey("B");
for (Map.Entry<String,Integer> e : map.entrySet())
    System.out.println(e.getKey() + "=" + e.getValue());
```

HashMap Internals (quick)

- Internally an array of **buckets**. Index = $\text{hash}(\text{key}) \& (n-1)$.

- On collision, entries form a **linked list**; if length > 8 and capacity >= 64, it converts to a **balanced tree** (Java 8+).
- Default capacity 16, load factor 0.75 — resizes (rehash) on exceeding threshold.
- Always override **both equals() and hashCode()** when using custom keys.

Comparable vs Comparator

```
class Employee implements Comparable<Employee> {
    int id; String name;
    public int compareTo(Employee o) { return this.id - o.id; } // natural order
}

// External comparator
Comparator<Employee> byName = (a, b) -> a.name.compareTo(b.name);
Collections.sort(list, byName);
```

13. Generics

Generics provide compile-time type safety and remove the need for casting.

```
class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}

Box<Integer> b = new Box<>();
b.set(10);
int x = b.get(); // no cast
```

- **Bounded type:** <T extends Number> — T must be Number or subclass.
- **Wildcards:** ? extends T (covariant — producer), ? super T (contravariant — consumer).
- **Type erasure** — generic type info is removed at runtime (replaced with raw type or bound).

14. Multithreading & Concurrency

A **thread** is the smallest unit of execution. Java has built-in concurrency support via the `java.lang.Thread` class and `java.util.concurrent` package.

Creating Threads

```
// 1. Extend Thread
class MyThread extends Thread {
    public void run() { System.out.println("Thread running"); }
}
new MyThread().start();

// 2. Implement Runnable (preferred)
Runnable r = () -> System.out.println("Runnable");
new Thread(r).start();

// 3. Callable (returns value, throws checked exception)
Callable<Integer> task = () -> 42;
ExecutorService es = Executors.newFixedThreadPool(2);
Future<Integer> f = es.submit(task);
int result = f.get();
es.shutdown();
```

Thread Lifecycle

- **NEW** — created but not started.
- **RUNNABLE** — eligible to run (running or ready).
- **BLOCKED** — waiting for monitor lock.
- **WAITING / TIMED_WAITING** — waiting indefinitely / for a time.
- **TERMINATED** — finished execution.

Synchronization

```
class Counter {  
    private int count = 0;  
    public synchronized void increment() { count++; } // method-level lock  
}  
  
// Block-level  
synchronized (lockObject) {  
    // critical section  
}
```

- **volatile** — guarantees visibility (no caching), but not atomicity.
- **Atomic classes** (AtomicInteger, AtomicLong) — lock-free atomic ops.
- **ReentrantLock** — explicit, more flexible than synchronized (tryLock, fair lock, condition).

Deadlock

Two or more threads waiting forever for locks held by each other. Avoid by acquiring locks in a fixed global order, using tryLock with timeout, or keeping critical sections short.

Executor Framework

- **Executors.newFixedThreadPool(n)** — fixed-size pool.
- **Executors.newCachedThreadPool()** — grows on demand.
- **Executors.newSingleThreadExecutor()** — single worker.
- **ScheduledExecutorService** — delayed / periodic tasks.
- Always call shutdown() or use try-with-resources (Java 19+).

15. Java 8+ Features

Lambda Expressions

```
// Old way  
Runnable r = new Runnable() {  
    public void run() { System.out.println("Hi"); }  
};  
  
// Lambda  
Runnable r2 = () -> System.out.println("Hi");  
  
// With params  
BiFunction<Integer,Integer,Integer> add = (a, b) -> a + b;
```

Functional Interfaces

Interface	Abstract Method	Use
Function<T,R>	R apply(T t)	Transform input → output
Predicate<T>	boolean test(T t)	Conditional check
Consumer<T>	void accept(T t)	Side effect
Supplier<T>	T get()	Provide a value
BiFunction<T,U,R>	R apply(T,U)	Two inputs

Stream API

```
List<Integer> nums = List.of(1,2,3,4,5,6);

int sumEvenSquares = nums.stream()
    .filter(n -> n % 2 == 0)
    .mapToInt(n -> n * n)
    .sum(); // 56

List<String> names = List.of("Ravi", "Anu", "Sam");
String csv = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.joining(", "));

Map<Boolean, List<Integer>> partitioned =
    nums.stream().collect(Collectors.partitioningBy(n -> n > 3));
```

- **Intermediate ops** (lazy): filter, map, sorted, distinct, limit, peek.
- **Terminal ops**: collect, forEach, count, reduce, findFirst, anyMatch.
- Streams are **not** data structures — they don't store data, they process it.
- Use **parallelStream()** cautiously; check actual performance gain.

Optional

```
Optional<String> opt = Optional.ofNullable(getName());
opt.ifPresent(System.out::println);
String n = opt.orElse("default");
String upper = opt.map(String::toUpperCase).orElse("NA");
```

Method References

- **Static**: Integer::parseInt
- **Instance of object**: System.out::println
- **Instance of type**: String::toUpperCase
- **Constructor**: ArrayList::new

16. File I/O & Serialization

Reading / Writing Files

```
// Read
try (BufferedReader br = Files.newBufferedReader(Path.of("data.txt"))) {
    String line;
    while ((line = br.readLine()) != null) System.out.println(line);
}

// Write
Files.writeString(Path.of("out.txt"), "Hello\n", StandardOpenOption.CREATE);
```

```
// One-shot
List<String> lines = Files.readAllLines(Path.of("data.txt"));
```

Streams Hierarchy

- **Byte streams** — `InputStream` / `OutputStream` (binary data).
- **Character streams** — `Reader` / `Writer` (text).
- Use **Buffered*** wrappers for performance.

Serialization

```
class User implements Serializable {
    private static final long serialVersionUID = 1L;
    String name; transient String password;    // transient skipped
}

// Write
try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("u.ser"))) {
    out.writeObject(user);
}
// Read
try (ObjectInputStream in = new ObjectInputStream(new FileInputStream("u.ser"))) {
    User u = (User) in.readObject();
}
```

17. JDBC — Database Access

JDBC (Java Database Connectivity) is the standard API to interact with relational databases.

Standard 6 Steps

- 1. Load driver (auto from Java 6+ via `ServiceLoader`).
- 2. Get a `Connection` via `DriverManager`.
- 3. Create a `Statement` / `PreparedStatement`.
- 4. Execute the query (`executeQuery` / `executeUpdate`).
- 5. Process the `ResultSet`.
- 6. Close resources (use `try-with-resources`).

```
String url = "jdbc:mysql://localhost:3306/mydb";
try (Connection con = DriverManager.getConnection(url, "user", "pass");
    PreparedStatement ps = con.prepareStatement(
        "SELECT id, name FROM users WHERE active = ?")) {
    ps.setBoolean(1, true);
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getInt("id") + " " + rs.getString("name"));
        }
    }
}
```

- **Statement** — basic, vulnerable to SQL injection.
- **PreparedStatement** — pre-compiled, safe, faster for repeats.
- **CallableStatement** — to call stored procedures.
- **Transactions** — `con.setAutoCommit(false); ... con.commit() / con.rollback();`

18. JVM Internals, Memory Model & Garbage Collection

JVM Memory Areas

- **Method Area / Metaspace** — class metadata, static fields, constant pool.
- **Heap** — all objects and instance variables. Shared across threads. Divided into **Young Gen** (Eden + S0 + S1) and **Old Gen**.
- **Stack** — per-thread; stores method frames, local variables, return addresses.
- **PC Register** — current executing instruction per thread.
- **Native Method Stack** — for native (JNI) calls.

Garbage Collection

- Java uses **automatic GC** based on the **generational hypothesis**: most objects die young.
- **Minor GC** — cleans Young Generation, fast.
- **Major / Full GC** — cleans Old Generation, slower, may pause app ("stop-the-world").
- Common collectors: **Serial**, **Parallel**, **CMS (deprecated)**, **G1 (default in Java 9+)**, **ZGC**, **Shenandoah**.
- You cannot force GC; `System.gc()` is only a hint.
- `finalize()` is deprecated — use `try-with-resources` or `Cleaner`.

Class Loaders (delegation hierarchy)

- **Bootstrap** — loads core JDK classes (rt.jar / java.base module).
- **Platform / Extension** — loads JDK platform classes.
- **Application / System** — loads classes from classpath.
- Class loading is on-demand and uses parent-first delegation.

19. L1 Interview Questions (50)

L1 questions test fundamentals. Aim for crisp, correct, 2–4 sentence answers.

Q. Why is Java platform-independent?

Java source is compiled to bytecode (.class), which is interpreted/JIT-compiled by the JVM. Since a JVM exists for every OS, the same bytecode runs anywhere — hence "Write Once, Run Anywhere".

Q. Difference between JDK, JRE, and JVM?

JDK = development kit (compiler + JRE + tools). JRE = runtime (JVM + libraries) to run programs. JVM = the engine that actually executes bytecode.

Q. Is Java pass-by-value or pass-by-reference?

Always pass-by-value. For objects, the value passed is a copy of the reference, so the called method can mutate the object but cannot reassign the caller's variable.

Q. Why is the main method static?

So the JVM can invoke it without creating an object of the class. It's the entry point.

Q. What is a constructor?

A special method with the same name as the class and no return type, invoked when an object is created. Used to initialize state.

Q. Default constructor — when provided?

If you don't define any constructor, the compiler inserts a public no-argument constructor that calls `super()`.

Q. Difference between `==` and `equals()`?

`==` compares references (or primitive values). `equals()` is a method that, by default, also compares references, but is overridden in classes like `String` and the wrappers to compare content.

Q. What is the String pool?

A special area of the heap that stores interned `String` literals. Two literals with the same content share the same object — saves memory.

Q. Why is String immutable?

For security (used as class names, file paths, etc.), thread-safety, hashCode caching (safe to cache), and to enable the String pool.

Q. Difference between `String`, `StringBuilder`, `StringBuffer`?

`String` is immutable. `StringBuilder` is mutable, non-synchronized (fast). `StringBuffer` is mutable and synchronized (thread-safe, slower).

Q. Difference between `final`, `finally`, and `finalize`?

`final` = modifier (constant variable / no-override method / no-extend class). `finally` = block that always runs after try-catch. `finalize()` = legacy method called by GC before reclaiming (deprecated).

Q. What is method overloading?

Same method name in the same class with different parameter lists. Resolved at compile time.

Q. What is method overriding?

Subclass redefines a parent method with same signature. Resolved at runtime — enables polymorphism.

Q. Can we override a static method?

No. Static methods are hidden (method hiding), not overridden — they're bound at compile time.

Q. Why doesn't Java support multiple inheritance of classes?

To avoid the Diamond Problem (ambiguity from two parents with the same method). Multiple inheritance is supported via interfaces.

Q. Difference between abstract class and interface?

Abstract class: can have state, constructors, both abstract and concrete methods. Interface: contract; only public methods (default/static allowed from Java 8), no instance state; supports multiple inheritance.

Q. What is encapsulation?

Bundling fields and methods together and restricting access using private fields with public getters/setters.

Q. What is polymorphism?

Same interface, different behavior. Two forms — compile-time (overloading) and runtime (overriding).

Q. What is abstraction?

Hiding implementation and exposing only essential behavior, achieved through abstract classes and interfaces.

Q. What is the difference between checked and unchecked exceptions?

Checked exceptions (extend `Exception`, not `RuntimeException`) must be caught or declared with `throws`. Unchecked (extend `RuntimeException`) don't need to be declared.

Q. Can a try block exist without a catch?

Yes — with a `finally` block, or as a `try-with-resources` statement.

Q. What is try-with-resources?

A `try` statement that declares resources implementing `AutoCloseable`; they are automatically closed when the block exits.

Q. Difference between throw and throws?

`throw` actually raises an exception inside code. `throws` declares in a method signature that the method may propagate listed checked exceptions.

Q. Why override hashCode() when you override equals()?

The contract says equal objects must have equal hash codes. Otherwise `HashMap`/`HashSet` will malfunction — duplicates may appear, lookups fail.

Q. Difference between ArrayList and LinkedList?

`ArrayList`: dynamic array, $O(1)$ random access, $O(n)$ middle insert. `LinkedList`: doubly linked, $O(1)$ insert/delete at ends, $O(n)$ random access.

Q. Difference between HashMap and Hashtable?

`HashMap` is not synchronized, allows one null key and many null values. `Hashtable` is synchronized (legacy), allows no nulls.

Q. Difference between HashMap and ConcurrentHashMap?

`HashMap` is not thread-safe. `ConcurrentHashMap` is — using bucket-level locking / CAS — and offers better scalability than synchronized maps.

Q. Difference between HashSet and TreeSet?

HashSet: unordered, $O(1)$. TreeSet: sorted (natural / Comparator), $O(\log n)$.

Q. What is Comparable vs Comparator?

Comparable defines natural ordering inside the class (compareTo). Comparator defines external orderings (compare) — you can have many.

Q. What is autoboxing?

Automatic conversion between primitives (int) and their wrapper objects (Integer) by the compiler.

Q. What is the Java Collections Framework?

A unified set of interfaces and classes (List, Set, Map, Queue and their implementations) for storing and manipulating groups of objects.

Q. What is the difference between fail-fast and fail-safe iterators?

Fail-fast iterators (ArrayList, HashMap) throw ConcurrentModificationException on structural modification during iteration. Fail-safe iterators (CopyOnWriteArrayList, ConcurrentHashMap) work on a snapshot and never throw.

Q. What is a wrapper class?

An object form of a primitive — Integer for int, Double for double, etc. Needed to use primitives where objects are required (e.g., collections).

Q. What is the difference between a static and an instance method?

Static methods belong to the class and can be called without an object. Instance methods require an object and can access instance state.

Q. What is the use of the static keyword?

Makes a member belong to the class rather than to instances — applies to variables, methods, blocks, and nested classes.

Q. What is a static block?

Executed once when the class is loaded — used to initialize static state.

Q. What does final mean for variables, methods, and classes?

Final variable = constant. Final method = cannot be overridden. Final class = cannot be extended.

Q. Difference between break, continue, return?

break exits the current loop/switch. continue skips to next iteration. return exits the method.

Q. What is varargs?

Variable-length arguments: declared as Type... name. Internally treated as an array.

Q. Can we have multiple public classes in one file?

No. Only one public top-level class is allowed per .java file, and its name must match the filename.

Q. What is the difference between heap and stack?

Stack stores method frames and local variables; each thread has its own. Heap stores all objects; shared by threads. Stack is fast and small; heap is large and GC-managed.

Q. What is Garbage Collection?

An automatic process that reclaims memory used by objects no longer reachable from any GC root. Java uses generational GC by default.

Q. Can we force Garbage Collection?

No. `System.gc()` is a hint to the JVM; whether it runs is implementation-dependent.

Q. What is an inner class?

A class declared inside another class. Types: static nested, non-static inner, local, anonymous.

Q. What is the difference between String literal and new String?

"Hello" uses the String pool; `new String("Hello")` creates a new object on the heap. `==` differs; `equals()` is true.

Q. What is the use of this keyword?

Refers to the current object. Used to disambiguate fields from parameters and to invoke another constructor of the same class via `this(...)`.

Q. What is the use of super keyword?

Refers to the parent class. Used to invoke parent constructor (`super(...)`) or call an overridden parent method (`super.method()`).

Q. What is the default value of an int, boolean, and reference variable?

`int` = 0, `boolean` = false, references = null (only for instance/static fields; locals are not auto-initialized).

Q. Can we declare a constructor as final?

No. Constructors cannot be inherited or overridden, so final is meaningless on them.

Q. Difference between Iterator and ListIterator?

Iterator works on any Collection, moves forward only. ListIterator works only on Lists, supports bidirectional traversal and modification via `add/set`.

20. L2 Interview Questions (40)

L2 questions probe deeper — internals, trade-offs, edge cases. Show reasoning, not just facts.

Q. Explain how HashMap works internally.

HashMap uses an array of buckets. For each key, it computes `hash = hash(key.hashCode())`, then `index = hash & (n-1)`. Collisions are chained in a linked list, which converts to a balanced tree when length > 8 and capacity ≥ 64 (Java 8+). On reaching the load factor (default 0.75), the table doubles and entries are rehashed.

Q. What happens if two keys have the same hashCode in a HashMap?

They land in the same bucket. The map then uses `equals()` to distinguish them; if `equals` returns true, the new value replaces the old, otherwise both coexist in the bucket's list/tree.

Q. Why must equals() and hashCode() always be overridden together?

Hash-based collections rely on this contract: `a.equals(b)` implies `a.hashCode() == b.hashCode()`. Violating it causes lost keys, duplicate entries, and broken lookups.

Q. Explain the Java Memory Model (JMM).

JMM defines how threads interact through memory: visibility of writes, ordering of operations, and the happens-before relationship. Without proper synchronization (`synchronized`, `volatile`, locks), one thread may not see another's writes.

Q. What is the volatile keyword and what does it guarantee?

`volatile` ensures visibility — reads always come from main memory and writes go to main memory immediately — and prevents reordering across the access. It does NOT make compound operations like `x++` atomic.

Q. What is the difference between synchronized and Lock (ReentrantLock)?

`synchronized` is implicit, block-scoped, simpler. `ReentrantLock` is explicit, supports `tryLock`, fairness policy, multiple Conditions, and interruptible locking — more flexible at the cost of manual unlock in finally.

Q. How does ConcurrentHashMap achieve thread safety?

Java 8 uses CAS for puts on empty buckets and synchronizes on the bucket's first node for collisions — much finer granularity than locking the whole table. Read operations are typically lock-free.

Q. What is a deadlock and how do you prevent it?

A cyclic wait where threads each hold a lock the other needs. Prevent by acquiring locks in a fixed global order, using `tryLock` with timeout, keeping critical sections short, or restructuring to avoid nested locks.

Q. Explain the difference between wait(), notify(), and notifyAll().

`wait()` releases the monitor and parks the thread until notified. `notify()` wakes one waiting thread; `notifyAll()` wakes all. They must be called while holding the object's monitor (inside `synchronized`).

Q. What is the difference between Callable and Runnable?

`Runnable.run()` returns void and cannot throw checked exceptions. `Callable.call()` returns a value and can throw checked exceptions. Callables are submitted to an `ExecutorService` and return a `Future`.

Q. What is the Executor Framework and why is it preferred over new Thread()?

It separates task submission from execution policy, reuses threads via pools, queues work, supports scheduling, exceptions, and graceful shutdown — far more scalable and manageable than spawning raw

threads.

Q. Difference between `CompletableFuture` and `Future`?

`Future` is a basic placeholder — you can only block on it via `get()`. `CompletableFuture` supports composing async pipelines with `thenApply/thenCompose/thenCombine`, exception handling, and combining many futures.

Q. Explain class loading in Java.

Class loaders use a parent-first delegation: Application → Platform → Bootstrap. Bootstrap loads core JDK classes; platform loads JDK platform modules; application loads user classes. Loading is lazy — on first reference.

Q. What are the phases of class loading?

Loading (read .class), Linking (verification, preparation, optional resolution), and Initialization (run static initializers and assignments).

Q. Explain Java memory areas (Heap, Stack, Metaspace).

Heap holds objects, shared, GC-managed; split into Young (Eden, S0, S1) and Old. Stack holds method frames and locals; per-thread. Metaspace (Java 8+) holds class metadata in native memory, replacing PermGen.

Q. What are types of GC in Java?

Serial (single-threaded), Parallel (throughput-oriented), CMS (concurrent, deprecated), G1 (default, region-based, mostly low-pause), ZGC and Shenandoah (low-latency, mostly concurrent).

Q. Strong, weak, soft, and phantom references — differences?

Strong = normal references (never GC'd while reachable). `SoftReference` = cleared only on memory pressure (cache-friendly). `WeakReference` = cleared at next GC (e.g., `WeakHashMap`). `PhantomReference` = used with `ReferenceQueue` for cleanup callbacks.

Q. What is a memory leak in Java?

Objects that are no longer needed but still strongly reachable, so GC cannot reclaim them. Common causes: static collections, unclosed listeners, `ThreadLocals`, cache without eviction.

Q. What is the difference between fail-fast and fail-safe iterators?

Fail-fast (`ArrayList`, `HashMap`) detect concurrent modification using a `modCount` and throw `ConcurrentModificationException`. Fail-safe (`CopyOnWriteArrayList`, `ConcurrentHashMap`) iterate over a snapshot or use weakly consistent iteration; they never throw.

Q. Explain generics type erasure.

Generic type parameters exist only at compile time. At runtime they are erased to their bound (`Object` by default). This is why you can't do `new T()` or `instanceof List<String>`.

Q. What is the difference between bounded and unbounded wildcards?

`<?>` is unbounded — any type. `<? extends T>` is upper-bounded (T or subtypes) — producers. `<? super T>` is lower-bounded (T or supertypes) — consumers. Remember PECS: Producer Extends, Consumer Super.

Q. Why is `String` final?

To preserve immutability — guarantees thread-safety, safe hashing/caching, secure use as identifiers, and the `String` pool. If `String` could be subclassed, those guarantees would break.

Q. What is the difference between a shallow copy and a deep copy?

Shallow copy duplicates the top-level object but shares referenced sub-objects. Deep copy duplicates the entire object graph so modifications don't affect the original.

Q. What is the role of serialVersionUID?

It's a version identifier for serialized classes. During deserialization, the JVM checks that the stream's UID matches the class's UID; mismatch throws `InvalidClassException`.

Q. Why is the transient keyword used?

To mark a field that should NOT be serialized — used for sensitive (password) or derived/non-serializable fields.

Q. How are streams lazy?

Intermediate operations don't execute on call — they build a pipeline. Execution starts only when a terminal operation runs, and elements flow through the pipeline one at a time (when possible).

Q. Stream vs parallelStream — when to use parallel?

Use `parallelStream` only when: data set is large, operations are CPU-bound and stateless, source supports cheap splitting (`ArrayList`, arrays), and you've measured a benefit. Avoid for small data, IO-bound work, or shared mutable state.

Q. What is the difference between map() and flatMap()?

`map(f)` applies `f` to each element producing one output per input. `flatMap(f)` applies `f`, where `f` returns a `Stream`, and concatenates them — useful for flattening collections of collections.

Q. What is the difference between intermediate and terminal stream operations?

Intermediate ops (`filter`, `map`, `sorted`) are lazy and return a new `Stream`. Terminal ops (`collect`, `forEach`, `reduce`, `count`) trigger execution and produce a result or side-effect.

Q. How do default methods affect interfaces?

Default methods let interfaces evolve without breaking implementers — they provide a default body. If a class inherits two conflicting defaults, it must override and explicitly choose (`Interface.super.method()`).

Q. What is the diamond problem and how does Java handle it?

When a class inherits the same method from two parents (an ambiguity). Java forbids multiple class inheritance and, for interfaces with conflicting defaults, requires the implementing class to override the method explicitly.

Q. What is the immutability checklist for a class?

Mark class `final`; make fields `private` and `final`; no setters; defensively copy mutable inputs/outputs; initialize all state in the constructor.

Q. Explain Singleton — and how to make it thread-safe.

A class with exactly one instance. Best implementations: (1) eager static `final` field; (2) lazy double-checked locking with a `volatile` field; (3) enum singleton — the cleanest, also handles serialization and reflection attacks.

Q. What is the Builder pattern? When to use?

A creational pattern that builds a complex object step by step using a separate `Builder` class with fluent setters and a `build()` method. Useful when a class has many optional parameters.

Q. What's the difference between composition and inheritance?

Inheritance is "is-a" — tightly coupled. Composition is "has-a" — loosely coupled. Prefer composition for flexibility and to avoid fragile base-class problems.

Q. Difference between checked and unchecked exceptions — when to use what?

Use checked for recoverable, anticipated failures (IO, SQL). Use unchecked for programming errors (NullPointerException, IllegalArgumentException). Don't wrap every checked exception in a RuntimeException blindly.

Q. What is the difference between Comparable and Comparator? Where do they go?

Comparable defines natural order inside the class (compareTo). Comparator defines external order in a separate class/lambda (compare). Use Comparator when you need multiple orderings or can't modify the class.

Q. How does try-with-resources work under the hood?

The compiler rewrites it into a try/finally that calls close() in reverse declaration order. If both the try and close throw, the close exceptions are added to the original as suppressed.

Q. What changes did Java 8 bring that affect API design?

Lambdas + functional interfaces enable concise behavior parameters. Default and static methods in interfaces let APIs evolve. Stream API replaces many imperative loops. Optional reduces null misuse. Date-Time API replaces the old Calendar.

Q. What is the difference between Process and Thread?

A process is an OS-level executing program with its own memory space. A thread is a unit of execution within a process; threads share the process's heap and code but have separate stacks.

21. Coding Problems for Practice

Common problems asked in screening rounds. Try to solve before reading the solution sketch.

Reverse a String

```
public static String reverse(String s) {
    return new StringBuilder(s).reverse().toString();
}
```

Check Palindrome

```
public static boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) if (s.charAt(i++) != s.charAt(j--)) return false;
    return true;
}
```

Find Largest in Array

```
public static int max(int[] a) {
    int m = a[0];
    for (int x : a) if (x > m) m = x;
    return m;
}
```

Fibonacci (iterative)

```
public static long fib(int n) {
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) { long t = a + b; a = b; b = t; }
    return a;
}
```

Factorial

```
public static long factorial(int n) {
    long r = 1;
    for (int i = 2; i <= n; i++) r *= i;
    return r;
}
```

Count Vowels and Consonants

```
public static int[] count(String s) {
    int v = 0, c = 0;
    for (char ch : s.toLowerCase().toCharArray()) {
        if (Character.isLetter(ch)) {
            if ("aeiou".indexOf(ch) >= 0) v++; else c++;
        }
    }
    return new int[]{v, c};
}
```

Check if a Number is Prime

```
public static boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; (long) i * i <= n; i++)
        if (n % i == 0) return false;
    return true;
}
```

Find Duplicates in Array

```
public static Set<Integer> duplicates(int[] a) {
    Set<Integer> seen = new HashSet<>(), dup = new HashSet<>();
    for (int x : a) if (!seen.add(x)) dup.add(x);
    return dup;
}
```

```
}
```

Find Missing Number 1..N

```
public static int missing(int[] a, int n) {  
    int sum = n * (n + 1) / 2;  
    for (int x : a) sum -= x;  
    return sum;  
}
```

Reverse a LinkedList

```
Node prev = null, curr = head;  
while (curr != null) {  
    Node next = curr.next;  
    curr.next = prev;  
    prev = curr;  
    curr = next;  
}  
head = prev;
```

First Non-Repeating Character

```
public static char firstUnique(String s) {  
    Map<Character,Integer> count = new LinkedHashMap<>();  
    for (char c : s.toCharArray()) count.merge(c, 1, Integer::sum);  
    for (var e : count.entrySet()) if (e.getValue() == 1) return e.getKey();  
    return '_';  
}
```

Check Two Strings are Anagrams

```
public static boolean isAnagram(String a, String b) {  
    if (a.length() != b.length()) return false;  
    char[] x = a.toCharArray(), y = b.toCharArray();  
    Arrays.sort(x); Arrays.sort(y);  
    return Arrays.equals(x, y);  
}
```

Count Word Frequency using Stream

```
Map<String,Long> freq = Arrays.stream(text.toLowerCase().split("\\W+"))  
    .filter(w -> !w.isEmpty())  
    .collect(Collectors.groupingBy(w -> w, Collectors.counting()));
```

Sort Map by Value

```
List<Map.Entry<String,Integer>> sorted = map.entrySet().stream()  
    .sorted(Map.Entry.<String,Integer>comparingByValue().reversed())  
    .collect(Collectors.toList());
```

Producer-Consumer with BlockingQueue

```
BlockingQueue<Integer> q = new ArrayBlockingQueue<>(10);  
new Thread(() -> { for (int i = 0; i < 100; i++) {  
    try { q.put(i); } catch (InterruptedException ignored) {}  
}}).start();  
new Thread(() -> {  
    while (true) {  
        try { System.out.println(q.take()); } catch (InterruptedException ignored) {  
            break; }  
    }  
}).start();
```

22. Quick Revision Cheat Sheet

Top 20 Things to Remember

- Java is pass-by-value — for objects, the value is a copy of the reference.
- String is immutable; live in the String pool; use StringBuilder for heavy edits.
- `==` compares references; `equals()` compares content (when overridden).
- Always override `hashCode()` with `equals()`. Same hash → same bucket.
- Method overloading = compile-time; overriding = runtime polymorphism.
- Java has no multiple class inheritance — only via interfaces.
- Interfaces (Java 8+) can have default and static methods; (Java 9+) private methods too.
- Checked exceptions must be handled or declared; unchecked extend `RuntimeException`.
- Use try-with-resources for any `AutoCloseable`.
- `ArrayList` for random access; `LinkedList` for frequent insert/delete.
- `HashMap` is not thread-safe; `ConcurrentHashMap` is.
- Default load factor = 0.75; tree-bucket threshold = 8.
- Fail-fast iterators throw `CME` on modification; fail-safe ones don't.
- `volatile` gives visibility, not atomicity. Use `Atomic*` or `synchronized` for compound ops.
- Prefer `ExecutorService` over `new Thread()`.
- Streams are lazy; terminal ops trigger execution.
- Use `Optional` to model "maybe a value" — avoid null pointer chains.
- `PreparedStatement` prevents SQL injection.
- Heap = objects; Stack = method frames; Metaspace = class metadata.
- G1 is the default GC since Java 9. ZGC/Shenandoah for ultra-low pause.

Time Complexities — Cheat

Operation	ArrayList	LinkedList	HashMap	TreeMap
get/contains	$O(1)$	$O(n)$	$O(1)$ avg	$O(\log n)$
add (end)	$O(1)$ amortized	$O(1)$	$O(1)$ avg	$O(\log n)$
add (middle)	$O(n)$	$O(n)$	—	—
remove	$O(n)$	$O(1)$ at end	$O(1)$ avg	$O(\log n)$

Interview Tips

- **Think out loud** — show your reasoning, not just the final answer.
- **Clarify the question** before coding — confirm input/output, constraints, edge cases.
- **Start brute force, then optimize** — interviewers value progression.
- **Mind edge cases** — empty input, single element, nulls, negatives, overflow.
- **Don't memorize, internalize** — understand the "why" of every concept.
- **Practice 1–2 problems daily** from LeetCode / HackerRank Easy & Medium.

- **Build a small project** (e.g., REST API with Spring Boot + MySQL) to anchor concepts.

All the best for your interviews! Consistency beats intensity — revise this guide daily for a week before D-Day.